Task-2:

Note: I was unable to find any functions for this so I simply did the math in an excel sheet, the .xlsx is also submitted along with this pdf please refer to it for values.

Questions:

What if total no. of threads exceed n?

Ans: Idle threads, we have threads which do nothing, provided that bounds checking is implemented. For example, if 1024 threads are occupied and size is 1000, we have 24 threads doing no work.

Why to check for bounds?

Ans: We check for bounds to ensure that threadIdx does not access any other memory other than the output memory, this might not lead to an explicit runtime error but instead might silently overwrite a memory location and we would have no idea(have not tested this). Also, from Task-2 most kernels are launched with idle threads, only 3 configs had just enough threads and rest had more than needed, thus better to bounds check always.

Which configurations failed/ behaved unexpectedly?

Ans:

Element-wise multiply and scale:

n = 10^5, block size = 32, no correct = 5530(varies each time but between 5500-5600)

n = 10^7, block size = 32, no correct = 560501(varies each time but around 5600500)

n = 10^7, block size = 128, no correct = 560458(varies each time but around 5600000)

n = 10^7, block size = 256, no correct = 560892

n = 10^7, block size = 512, no correct = 561439

One thing I feel is that there is a lot of approximation here, when I allow deviation of e^{-1}, then all cases pass for all configs. The above values are for deviation of e^{-3}.

On a side-note for element wise scale and multiply, at first, I used equal to and then switched to epsilon of 1e-3. For others I used equal to which worked well enough.

ReLU:

All configs passed for relu, there was no need for any epsilon a simple equal to was enough.

Vector Addition:

All configs pass for vector addition with a simple equal to as well.

Smaller block size vs larger? Which is better? What are the trade-offs?

Ans:

Throughout task-2 in only one place could I see any difference, in element wise multiply and scale for 10^5 input size and 32 as block size not all cases passed while every other given config passed with epsilon of 1e-3, I also tried out block size of 64 which failed similarly, hence I am prompted to say that a higher block size means lesser approx. calculations from this empirical evidence. And quite obviously from the number

of threads launched(.xlsx file), a larger block size would mean more idle threads are launched.

Task-3:

This was a bit confusing.

As block size decreased the time seemed to decrease slightly from 512→256→128 but then increases for 32 and the timings varied wildly for the same size from 1.20 ms to 0.15 ms and sometimes even 7ms popped up. Though, 7ms was the one which popped up for the 1st execution and then repeated executions took much less time like 0.18 ms.

I used cudaEvent_t and functions to calculate time. This code is available separately as vecAddEvent.cu.